Module 3: Pandas & Matplotlib

A Comprehensive Academic Reference

May 11, 2026

1 Introduction to Pandas

Pandas is a Python library for data manipulation and analysis, built on NumPy. Its two core data structures are the **Series** (one-dimensional labelled array) and the **DataFrame** (two-dimensional tabular structure). A Series can hold any data type and aligns data by index during arithmetic operations. A DataFrame is effectively a dictionary of Series sharing a common index.

1.1 Series and DataFrame Creation

```
import pandas as pd

# Series: 1D labelled array
grades = pd.Series([85, 92, 78, 95], name="Score")
grades_idx = pd.Series([85, 92, 78], index=["Alice", "Bob", "Carol"])

# DataFrame: 2D table from a dict
df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Carol"],
    "Score": [85, 92, 78],
    "Passed": [True, True, False]
})
```

1.2 Reading Data with `read_csv()`

The `read_csv()` function imports structured files into a DataFrame with extensive options for real-world data quirks.

```
df = pd.read_csv("data.csv")
# Common options:
df = pd.read_csv("data.csv", sep=";", header=0, index_col=0,
    usecols=["Name","Age"], dtype={"Age": int},
    na_values=["NA","null"], skiprows=2, nrows=100)
```

Other readers include `read_excel()`, `read_json()`, `read_sql()`, and `read_parquet()`.

1.3 Inspecting Data

```
df.head()       # First 5 rows; head(10) for 10
df.tail(3)      # Last 3 rows
df.info()       # Columns, non-null count, dtypes, memory usage
df.describe()   # count, mean, std, min, 25%, 50%, 75%, max
df.shape        # (rows, columns) tuple
df.columns      # Column labels
df.dtypes       # Data type per column
```

`info()` reveals missing values (via non-null counts); `describe()` gives summary statistics for numeric columns.

2 Selecting and Filtering Data

Pandas offers multiple indexing mechanisms. The critical distinction is between label-based selection (`loc[]`) and position-based selection (`iloc[]`).

2.1 Column Selection

Single columns return a Series; multiple columns (via a list inside brackets) return a DataFrame.

```
df["Name"]            # Series
df[["Name", "Score"]] # DataFrame (double brackets)
```

2.2 Label-Based Selection: `loc[]`

`loc[]` selects by index label and column label. Slices are **inclusive** on both endpoints.

```
df = pd.DataFrame({"Score": [85, 92, 78, 95]},
                  index=["Alice", "Bob", "Carol", "David"])

df.loc["Alice"]                  # Single row
df.loc["Alice":"Carol"]          # Alice, Bob, Carol (inclusive)
df.loc[["Alice", "David"]]       # Specific rows
df.loc["Alice":"Carol", ["Score"]] # Row slice + column
df.loc[df["Score"] > 85]         # Conditional selection
```

2.3 Position-Based Selection: `iloc[]`

`iloc[]` selects by integer position with standard Python slicing rules: the end is **exclusive**.

```
df.iloc[0]        # First row (position 0)
df.iloc[1:4]      # Rows 1, 2, 3 (NOT 4)
df.iloc[0:3, 0]   # First 3 rows, column 0
df.iloc[[0, 2, 4]] # Rows at positions 0, 2, 4
```

2.4 Boolean Filtering

Boolean indexing filters rows by evaluating a condition on each element.

```
df = pd.DataFrame({"Name": ["Alice","Bob","Carol","David","Eve"],
                   "Age": [25, 30, 22, 35, 28],
                   "Score": [85, 92, 78, 95, 88]})

adults = df[df["Age"] >= 25]
# Multiple conditions: MUST wrap each in parentheses
high_scorers = df[(df["Age"] >= 25) & (df["Score"] > 85)]
young_or_high = df[(df["Age"] < 25) | (df["Score"] > 90)]
```

Critical: Each condition must be wrapped in parentheses when combining with `&` or `|`. These bitwise operators have higher precedence than comparisons. Using Python's `and`/`or` keywords fails because they do not operate element-wise on Series.

3 Data Cleaning

Real-world datasets contain missing values, inconsistencies, and errors. Pandas provides systematic tools for detection and remediation.

3.1 Detecting Missing Data

`isnull()` returns a Boolean mask; `.sum()` counts missing values per column.

```
df = pd.DataFrame({"Name": ["Alice","Bob",None,"David"],
                   "Age": [25, None, 22, 35],
                   "Score": [85., 92., None, 95.]})

df.isnull()              # Boolean mask (True = NaN)
df.isnull().sum()        # Missing count per column
df.isnull().sum().sum()  # Total missing values
```

3.2 Removing Missing Data: `dropna()`

```
df.dropna()               # Drop any row with any NaN
df.dropna(axis=1)         # Drop any column with any NaN
df.dropna(how="all")      # Drop rows where ALL values are NaN
df.dropna(thresh=2)       # Keep rows with >= 2 non-null values
df.dropna(subset=["Age"]) # Drop rows where Age is NaN
```

3.3 Filling Missing Data: `fillna()`

When dropping data is undesirable, imputation preserves observations.

```
df.fillna(0)                        # Constant fill
df.fillna(method="ffill")           # Forward fill (propagate last valid)
df.fillna(method="bfill")           # Backward fill (next valid)
df["Age"].fillna(df["Age"].mean()) # Mean imputation
df["Score"].fillna(df["Score"].median()) # Median (outlier-robust)
```

3.4 Complete Cleaning Workflow

```
df = pd.DataFrame({"Name": ["Alice","Bob",None,"David","Eve"],
                   "Age": [25, None, 22, 35, 28],
                   "Income": [55000,62000,None,95000,48000],
                   "Category": ["A","B","A",None,"B"]})
# 1. Diagnose
print(df.isnull().sum())
# 2. Drop rows missing critical categories
df = df.dropna(subset=["Name", "Category"])
# 3. Impute numeric columns with median
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Income"] = df["Income"].fillna(df["Income"].median())
# 4. Reset index for contiguous integer indexing
df = df.reset_index(drop=True)
```

4 Sorting, Grouping, and Transforming

After cleaning, analysis proceeds through sorting, grouping, and transformation — operations inspired by SQL and functional programming.

4.1 Sorting with `sort_values()`

```
df = pd.DataFrame({"Name": ["Alice","Bob","Carol","David"],
                   "Score": [85, 92, 78, 95], "Age": [25, 30, 22, 35]})

df.sort_values("Score")                    # Ascending
df.sort_values("Score", ascending=False)   # Descending
df.sort_values(["Age","Score"], ascending=[True,False]) # Multi-key
df.sort_values("Score", na_position="first") # NaN first
```

4.2 Grouping and Aggregation with `groupby()`

The split-apply-combine paradigm: data is split by keys, a function is applied to each group, and results are combined.

```
df = pd.DataFrame({"Dept": ["Sales","Sales","HR","HR","IT","IT"],
                   "Emp":   ["Alice","Bob","Carol","David","Eve","Frank"],
                   "Salary": [55000,62000,48000,52000,75000,71000],
                   "Years": [3,5,2,4,6,4]})

# Single aggregation returns a Series with Dept as index
df.groupby("Dept")["Salary"].mean()

# Multiple aggregations with agg()
df.groupby("Dept").agg({"Salary": ["mean","min","max"],
                        "Years": ["mean","std"]})

# Named aggregations (flat column names)
df.groupby("Dept").agg(
    avg_sal=("Salary","mean"),
    count=("Emp","count"),
    max_yr=("Years","max")
).reset_index()  # <-- Restore Dept as a regular column
```

Important: GroupBy results place grouping columns as the index. Use `reset_index()` to convert them to regular columns.

4.3 `apply()` vs. Vectorized Operations

`apply()` iterates in Python and is slow on large datasets. Vectorized operations (NumPy, Pandas built-ins) are orders of magnitude faster and should always be preferred when available.

```
# apply() -- flexible, slow
df["Name_Len"] = df["Name"].apply(len)

# Vectorized equivalent -- much faster
import numpy as np
df["Band"] = np.where(df["Salary"] < 50000, "Low",
             np.where(df["Salary"] < 70000, "Medium", "High"))
```

4.4 Adding and Modifying Columns

```
df["Bonus"] = df["Salary"] * 0.10        # Simple derived column
df["Total"] = df["Salary"] + df["Bonus"]  # Composite column

# Conditional column creation
conditions = [df["Years"]<3, df["Years"]<5, df["Years"]>=5]
choices = ["Junior","Mid-level","Senior"]
df["Level"] = np.select(conditions, choices, default="Unknown")

df = df.drop("Bonus", axis=1)             # Drop column
df = df.rename(columns={"Years": "Tenure"}) # Rename
```

5 Matplotlib Basics

Matplotlib is Python's foundational plotting library. The `pyplot` module (`plt`) provides a MATLAB-like interface. A **Figure** is the top-level container (the entire window or page); an **Axes** is a plotting area with x-axis, y-axis, data, labels, and title. A single figure can contain multiple axes.

5.1 The Figure and Axes Model

The **object-oriented style** (`fig, ax`) is preferred over the state-based interface for unambiguous control.

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot([1, 2, 3, 4], [1, 4, 9, 16])
ax.set_title("Square Numbers"); ax.set_xlabel("x"); ax.set_ylabel("y")
plt.show()
```

5.2 Line Plots

Line plots visualize relationships between continuous variables with extensive marker, style, and color customization.

```
x = np.linspace(0, 10, 50)
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, np.sin(x), marker="o", linestyle="-", color="darkblue",
       markersize=4, linewidth=1.5, label="sin(x)")
ax.plot(x, np.cos(x), marker="s", linestyle="--", color="darkred",
       markersize=4, linewidth=1.5, label="cos(x)")
ax.set_title("Trigonometric Functions", fontsize=14, fontweight="bold")
ax.set_xlabel("x (radians)"); ax.set_ylabel("f(x)")
ax.legend(loc="upper right"); ax.grid(True, alpha=0.3)
plt.show()
```

5.3 Bar Plots

Grouped bars are created by offsetting x-positions by a fraction of the bar width.

```
cats = ["Q1","Q2","Q3","Q4"]
r23 = [120, 135, 150, 180]; r24 = [140, 155, 170, 210]
x = np.arange(len(cats)); w = 0.35

fig, ax = plt.subplots(figsize=(8, 5))
b1 = ax.bar(x-w/2, r23, w, label="2023", color="steelblue", edgecolor="black")
b2 = ax.bar(x+w/2, r24, w, label="2024", color="coral", edgecolor="black")
for bars in [b1,b2]:
    for bar in bars:
        ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
                str(bar.get_height()), ha="center", fontsize=8)
ax.set_title("Quarterly Revenue", fontsize=14, fontweight="bold")
ax.set_xticks(x); ax.set_xticklabels(cats)
ax.legend(); ax.set_ylim(0,250); plt.show()
```

6 Additional Visualization Types

6.1 Histograms

A histogram bins a continuous variable and displays frequency or density. The `bins` parameter is critical: too few bins mask features, too many introduce noise.

```
np.random.seed(42)
data = np.random.normal(loc=70, scale=15, size=500)

fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(data, bins=25, color="steelblue", edgecolor="white",
       alpha=0.85, density=True)
# Overlay theoretical PDF
from scipy import stats
x_fit = np.linspace(data.min(), data.max(), 200)
ax.plot(x_fit, stats.norm.pdf(x_fit,70,15), color="darkred", lw=2, label="N(70,15)")
ax.set_title("Distribution of Scores", fontsize=14, fontweight="bold")
ax.set_xlabel("Score"); ax.set_ylabel("Density"); ax.legend(); plt.show()
```

6.2 Scatter Plots

Scatter plots reveal correlations and clusters. Color mapping and transparency encode additional dimensions.

```
np.random.seed(42)
h = np.random.normal(170, 10, 150)       # height
w = h * 0.45 + np.random.normal(0, 8, 150) # weight
age = np.random.randint(18, 65, 150)

fig, ax = plt.subplots(figsize=(8, 5))
sc = ax.scatter(h, w, c=age, cmap="viridis", s=60, alpha=0.7,
               edgecolor="gray", linewidth=0.3)
cbar = fig.colorbar(sc); cbar.set_label("Age", rotation=270, labelpad=15)
ax.set_title("Height vs. Weight by Age", fontsize=14, fontweight="bold")
ax.set_xlabel("Height (cm)"); ax.set_ylabel("Weight (kg)")
ax.grid(True, alpha=0.2); plt.show()
```

6.3 Subplots

`plt.subplots(nrows, ncols)` returns a figure and an array of axes for multi-panel layouts.

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0,0].plot(x, np.sin(x), color="steelblue")
axes[0,0].set_title("Line")
axes[0,1].bar(["A","B","C"],[23,17,35], color="coral")
axes[0,1].set_title("Bar")
axes[1,0].hist(data, bins=20, color="seagreen", edgecolor="white")
axes[1,0].set_title("Histogram")
axes[1,1].scatter(h, w, c=age, cmap="viridis", alpha=0.6, s=30)
axes[1,1].set_title("Scatter")
fig.suptitle("Dashboard", fontsize=16, fontweight="bold")
plt.tight_layout(); plt.show()
```

6.4 Saving Figures

Use `fig.savefig()` or `plt.savefig()` to export to PNG, PDF, or SVG. **Always save before calling `show()`**; in most backends, `show()` clears the figure.

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, np.sin(x), label="sin(x)"); ax.legend()
fig.savefig("plot.png", dpi=300, bbox_inches="tight") # Print-quality PNG
fig.savefig("plot.pdf", bbox_inches="tight")       # Vector PDF
plt.show() # After saving!
```

7 Common Gotchas and Summary

7.1 SettingWithCopyWarning

This warning fires when modifying through chained indexing, where Pandas cannot determine whether the assignment targets the original or a copy.

```
# BAD: chained indexing -- may not modify original
df[df["A"] > 0]["B"] = 0
# GOOD: use .loc for unambiguous assignment
df.loc[df["A"] > 0, "B"] = 0
```

Rule: Always use `.loc[]` for assignments that depend on a condition.

7.2 `loc[]` vs `iloc[]`: Inclusive vs Exclusive

- `loc[start:end]` — **inclusive** on both endpoints (label semantics).
- `iloc[start:end]` — **exclusive** on the end (Python convention).

```
df = pd.DataFrame({"V": [10,20,30,40,50]}, index=["a","b","c","d","e"])
df.loc["a":"c"] # a, b, c (3 rows, inclusive)
df.iloc[0:3]    # rows 0,1,2 (exclusive end, same 3 rows here)
```

7.3 `plt.show()` Resets the Figure

`show()` clears the figure by design. Save before displaying.

```
# WRONG: saves blank
plt.plot([1,2,3],[1,2,3]); plt.show(); plt.savefig("plot.png")
# CORRECT
plt.plot([1,2,3],[1,2,3]); plt.savefig("plot.png"); plt.show()
```

7.4 GroupBy Results Need `reset_index()`

After `groupby().agg()`, grouping columns become the index. Call `reset_index()` to restore them as regular columns.

```
g = df.groupby("Dept")["Salary"].mean()    # index = Dept
g_flat = df.groupby("Dept")["Salary"].mean().reset_index() # Dept is a column
```

7.5 Summary of Key Concepts

Concept	Key Point
Series & DataFrame	Series is 1D labelled array; DataFrame is 2D table with labelled axes.
`read_csv()`	Handles delimiters, headers, types, and missing-value markers.
`loc[]` vs `iloc[]`	loc: label-based, inclusive slice. iloc: position-based, exclusive slice. Assign via loc to avoid SettingWithCopyWarning.
Boolean filtering	Wrap each condition in `()` when using `&` or `|`.
Missing data	`isnull().sum()` detects; `dropna()` removes; `fillna()` imputes.
`groupby().agg()`	Split-apply-combine; use `reset_index()` to flatten output.
`apply()` vs vectorized	Vectorized (NumPy) operations are orders of magnitude faster.
Matplotlib model	Figure = container; Axes = plot area. Use `fig, ax = plt.subplots()`.
Save figures	Call `savefig()` **before** `show()`; `bbox_inches="tight"`.
Subplots	`plt.subplots(nrows, ncols)` returns figure and grid of axes.

Mastery of Pandas and Matplotlib constitutes the core workflow of exploratory data analysis in Python. Pandas handles ingestion, cleaning, transformation, and aggregation, while Matplotlib creates publication-quality visualizations to communicate findings. Their integration forms the foundation for advanced topics including machine learning pipelines and interactive dashboards.